FUSE Narrow Gauge User Manual

Narrow-gauge and dual-gauge track for Railroader

Contents

Getting Started

Requirements

- Railroader `2025.1.x`
- Unity Mod Manager
- FUSE — required, and Narrow Gauge loads after it

Install

1. Install Unity Mod Manager for Railroader.
2. Install FUSE and confirm it loads.
3. Place the `FUSE.NarrowGauge` folder in `Railroader/Mods`.
4. Start Railroader and load a map.

Verify

```
/fuse.loaded
```

Both `FUSE` and `FUSE.NarrowGauge` should be listed and applied.

What It Does

Narrow Gauge is a capability module, not a content pack. It lets route packages author narrow-gauge and dual-gauge track, and it renders the specialised trackwork that goes with them: frogs, guard rails, wing rails, blades, and crossings.

On a route that doesn't use narrow gauge, nothing changes. If you installed it expecting to see narrow-gauge track appear, that is why nothing looks different.

What it handles for the route author:

- **Generated narrow routing.** A dual-gauge segment automatically gets a real native narrow-gauge ghost route. Authors do not place ghost track.
- **Measured trackwork geometry.** Validated plans replace fixed dual-gauge turnout rails with calculated running rails, frog noses, wing rails, and guard rails. Invalid plans fall back to the existing geometry rather than breaking.
- **Automatic diamonds.** Two standard-gauge segments crossing at the same grade without being joined are compiled into a fixed diamond crossing on their own.
- **Hidden generated scaffolding.** Ghost rails, roadbed, masks, and bumpers stay hidden; a generated ghost switch shows only when it owns a real narrow-to-dual transition.

Debug View

The Unity Mod Manager panel can enable a special-work debug view, which colours the compiler's understanding of the track:

Colour	Meaning
Blue	Standard routes and physical rails
Cyan	Narrow routes and physical rails
Green	Shared rails
Orange	Intersections
Red	Frog candidates
Purple	Blades and stub assemblies

This is an authoring and diagnostic tool — leave it off during normal play.

Multiplayer And Saves

Narrow Gauge builds on FUSE, so the same expectations apply: every player needs the same mods and package list installed locally. Back up saves before adding or removing any world-changing mod.

Reporting Problems

Include `FUSE.log`, `Player.log`, `/fuse.loaded` output, the route package involved, and a screenshot of the trackwork if it is a geometry problem. Enabling the debug view before the screenshot makes a geometry report far more useful.

Issues: <https://github.com/Hrogers-Rog/Narrow_Gauge/issues>

Next

- Authoring Guide — building narrow and dual-gauge track
- Special-Work Catalog — the preset list

Authoring Guide

How to build narrow-gauge and dual-gauge track in a FUSE route package.

The workflow is: **place normal track, assign segment gauges, then bind a preset to one anchor node.** Narrow Gauge derives the routes, physical rails, intersections, frogs, guards, blades, and native topology from the surrounding track — you do not hand-place any of it.

Segment Gauges

Set `gauge` on FUSE `tracks.segments` entries:

Value	Meaning
`Narrow`	Narrow gauge only
`DualGauge`	Dual gauge, shared rail side inferred
`DualGauge_L`	Dual gauge, shared rail is the left standard rail
`DualGauge_R`	Dual gauge, shared rail is the right standard rail
`DualGauge_T`	Dedicated shared-rail transition segment

Omitting `gauge` leaves the segment standard.

Dual-Gauge Rules

A `DualGauge` segment **automatically receives a real native narrow-gauge ghost route** with deterministic `fuse-ng:*` ids. Do not author ghost track yourself.

`DualGauge_L` and `DualGauge_R` explicitly select which outer standard rail is shared. Left and right refer to the physical frame authored on the track nodes, so reversing a segment's `startId` and `endId` does **not** swap the selected rail — this is deliberate, and it is the thing most likely to surprise you if you think of left/right as direction-relative.

Plain `DualGauge` infers the shared side from narrow branch transitions and propagates it through connected dual segments. Explicit `fuse-ng:shared-rail=left|right` tags still work. An unconstrained dual-gauge component defaults to the right rail.

`DualGauge_T` marks a degree-two transition between a `DualGauge_L` and a `DualGauge_R` segment, where the shared rail flips sides. Its generated narrow route shifts between the neighbouring offsets. This is catalog preset `dual.shared-rail-flip`; it is segment-anchored and generates automatically when the marked segment has exactly one `DualGauge_L` and one `DualGauge_R` neighbour.

Switches

Fully dual-gauge turnouts mechanically synchronise their standard and generated narrow switch routes. Route mapping follows linked segments rather than assuming both native switches share the same normal/reversed ordering.

A switch with two `DualGauge` main legs and one `Narrow` branch compiles automatically into a fixed standard route plus a real three-leg narrow switch on the generated narrow graph. Standard equipment cannot route onto the narrow-only branch.

Selecting Special Work

Bind a preset to one existing anchor node through the `narrowGauge.specialWork` extension. FUSE itself needs no Narrow Gauge schema or patch.

```
{
  "extensions": {
    "narrowGauge.specialWork": {
      "version": 1,
      "objects": {
        "yard-narrow-lead": {
          "preset": "dual.narrow-branch-joins-main",
          "anchorNode": "NCustom_0ifg",
          "parameters": {
            "sharedRailSide": "auto",
            "frogNumber": 8
          }
        }
      }
    }
  }
}
```

`objects` also accepts an array of objects with explicit `id` properties.

Unknown presets, incompatible topology, duplicate ids, and duplicate anchor bindings are logged as authoring errors rather than silently ignored. Check `FUSE.log` when a preset does not appear.

Preset Families

The full list with topology contracts is in special-work-catalog.md.

Family	Examples
Turnouts	`turnout.standard.left/right/wye`, `turnout.narrow.*`
Dual-gauge turnouts	`dual.narrow-branch-joins-main`, `dual.standard-branch-joins-main`, `dual.both-diverge`, `dual.split-standard-narrow`
Three-way	`three-way.standard`, `three-way.narrow`, `three-way.dual`
Crossings	`crossing.diamond`, `crossing.arbitrary-angle`, `crossing.90-degree`
Slips	`slip.single`, `slip.double`
Stubs	`stub.left`, `stub.right`, `stub.three-way`
Transitions	`dual.shared-rail-flip`

Automatic Diamonds

Fixed standard-gauge diamonds are the exception to the anchor-node workflow. Two sufficiently long standard segments that cross at the same grade **without being joined** are detected directly — they need neither an anchor node nor ghost nodes.

The source routes stay graph-disconnected; only their rail and tie rendering is replaced through the measured crossing envelope. Skew diamonds classify the four physical intersections as two outward acute V-frogs and two inner obtuse K-frogs.

Verifying Your Work

1. Load the route and run `/fuse.loaded` to confirm the package applied.
2. Check `FUSE.log` for authoring errors from the special-work parser.
3. Enable the debug view in the Unity Mod Manager panel and look at the colours — cyan narrow routes, green shared rails, orange intersections, red frog candidates. If a frog candidate is missing or in the wrong place, the topology is not what the preset expected.
4. Run a train over it.

Limits

- **Three-way switches** are in the catalog as staged topology, but Railroader does not support them as ordinary graph switches.
- **Cross-family coupling** is not active. `DualGaugeLinkRegistry.CanBridgeCoupling(...)` is the boundary where that permission will live, but no patch enforces it yet.
- **Invalid measured plans** fall back to existing fallback geometry rather than failing the load — so bad geometry shows up as "it looks like it used to," not as an error.

Related

- Special-Work Catalog
- Getting Started
- FUSE Package Author Guide

Special-Work Catalog

The catalog identifies reusable special-work types from authored gauge markers and measured topology. Runtime matching must not depend on a particular `NCustom_*` or `SCustom_*` instance ID.

Catalog Model

Anchor	Used For	Recognition
Node	Turnouts, wyes, slips, and multi-route junctions	Connected route families, switch states, intersections, and measured rail geometry
Segment	Fixed transitions and future segment-based crossings	Segment marker, endpoint degree, neighboring track families, and measured rail geometry

Truth tables are appropriate when a type has movable blades, route-state ownership, or compound frog/rail ownership that must be validated. Fixed transitions use a segment topology contract instead.

Transitions

`dual.shared-rail-flip`

Fixed transition that moves the narrow-gauge shared rail from one standard outer rail to the other.

Recognition contract:

- The anchor segment is marked `DualGauge_T`.
- Both anchor endpoints are degree two after generated and hidden control segments are excluded.
- Each endpoint has exactly one non-transition dual-gauge neighbor.
- The two neighbors are one explicitly marked `DualGauge_L` and one explicitly marked `DualGauge_R`.
- Neighbor order is irrelevant. Both L-to-R and R-to-L generate the same procedural type.

Generated hardware:

- Two continuous standard running rails.
- Two measured narrow transition rails.
- Two tapered shared-rail frog points.
- Four guard rails.
- Dual-gauge transition ties and collider.

Invalid topology renders the normal dual-gauge fallback and logs the failed contract. This transition has no movable blades or switch state, so it does not use a turnout truth table.

Switches And Wyes

Preset	Family
`turnout.standard.left`	Standard turnout
`turnout.standard.right`	Standard turnout

Preset	Family
`turnout.standard.wye`	Standard wye
`turnout.narrow.left`	Narrow turnout
`turnout.narrow.right`	Narrow turnout
`turnout.narrow.wye`	Narrow wye
`dual.narrow-branch-joins-main`	Dual-gauge turnout
`dual.standard-branch-joins-main`	Dual-gauge turnout
`dual.both-diverge`	Dual-gauge turnout
`dual.split-standard-narrow`	Dual-gauge split
`three-way.standard`	Standard three-way
`three-way.narrow`	Narrow three-way
`three-way.dual`	Dual-gauge three-way

Crossings And Slips

Preset	Family
`crossing.diamond`	Diamond crossing
`crossing.arbitrary-angle`	Measured-angle crossing
`crossing.90-degree`	Right-angle crossing
`slip.single`	Single slip
`slip.double`	Double slip

`crossing.diamond` is active for isolated same-grade intersections between two ordinary standard-gauge segments. Discovery rejects shared endpoints, grade-separated routes, angles below 8 degrees, crossings without enough lead for all four physical frogs, and nearby compound crossing zones. The native segments are never connected to each other at the geometric intersection. A skew diamond renders two outer acute V-frogs, two inner obtuse/K frogs, and one derived check rail per running rail.

Stub Switches

Preset	Family
`stub.left`	Left stub turnout
`stub.right`	Right stub turnout
`stub.three-way`	Three-way stub turnout